

PW 1: Basic Python Instructions

Goals

- Know how to use the Python interpreter and the Visual Studio Code development environment.
- Understand and know how to correct Python programs.
- Know how to define and use variables.
- Know how to use control structures (sequence, conditional, loops).

We will use the Visual Studio Code for Ubuntu. Visual Studio Code is available for free from the following link: <https://code.visualstudio.com/download>. Also we can use Ubuntu terminal to run python codes.

Exercise 1: Python on Ubuntu terminal

Open the Terminal and type some commands (for example some arithmetic operations, definition of variables ...).

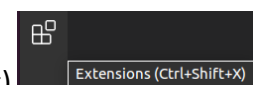
To open Ubuntu terminal, you can:

- Go to Ubuntu toolbar, click on the “Show Applications” button then search for the terminal and open it.
- On your keyboard, tap ctrl+alt+t at simultaneously.

Exercise 2:

Visual Studio Code is an integrated development environment for Python (and other programming languages). Its interface is shown in figure 1. It offers an editor (right part), a terminal to run programs and type Python instructions in interactive mode (bottom right) and a directory window at the left. Also, a toolbar on the top.

1. To download the python package click on the extensions icon (left)
2. Search for ms-python.python and then download it
3. Try some python instructions in the terminal.



4. After downloading the PW1 folder from moodle, use the toolbar File/Open folder to open the PW1 folder (don't forget to extract PW1 before you open it). It should appear in the editor on the left.
5. Open the perimeter_circle.py file. It should appear in the editor on the right.
6. Run the program by clicking on the green play button. It runs in the terminal and asks to enter the radius. Enter a value. The perimeter should be displayed.

Exercise 3: Correct the program

The following program contains some errors

```
# no comments or significant identifiers
def main():
    print('start')
    x == 0;
    while x < 3
        print(x)
        if x = 1
            print('1')
    x += 1
    print(end)

main
```

1. Find the errors and specify whether they are syntax errors or semantic errors. Will they be indicated when the program is read or when it is executed. You can refer to the following resource to have an idea about different types of errors in python:
<https://www.coursera.org/lecture/python-basics/syntax-runtime-and-semantic-errors-4f9q0>
2. Correct the program.

Exercise 4: Understand the program

What is the result of the execution of the following program.

```
# voluntarily, no comments or significant identifiers  
# voluntarily, the identifiers are insignificant.
```

```
def prog1():  
    for n in range(0, 9):  
        if n % 2 == 0:  
            print(n, 'divisible by 2')  
        if n % 3 == 0:  
            print(n, 'divisible by 3')  
  
def prog2():  
    for n in range(0, 9):  
        if n % 2 == 0:  
            print(n, 'divisible by 2')  
        if n % 3 == 0:  
            print(n, 'divisible by 3')  
  
def prog3():  
    for n in range(0, 9):  
        if n % 2 == 0:  
            print(n, 'divisible by 2')  
        elif n % 3 == 0:  
            print(n, 'divisible by 3')  
  
print('prog1 :')  
prog1()  
  
print('prog2 :')  
prog2()  
  
print('prog3 :')  
prog3()
```

Exercise 5:

Write a program that calculates P / P_t for γ and ϕ entered by the user. No control will be made on the entry. Use the formula given below and test the program for $\gamma = 1.4$ and $\phi = 0.5$. The expected

- The program will ask the user 10 questions from the selected multiplication table in a random way between 1 and 10. To obtain a random number, we will use the *randint* function of the random module. Do help ('*random.randint*') under the Python interpreter to get its documentation.
- Each time the program asks, the user should answer and the program is counting in the background the number of correct answers.
- At the end of the test/revision, the program will show:
 - "Excellent!" if all the answers are correct,
 - "Very good." If he made only one mistake,
 - "Good." If he made less than three mistakes,
 - "Average." If he has 4, 5 or 6 correct answers,
 - "You should rework this table." If he has 3 correct answers or less
 - "You MUST rework this table." If all his answers are wrong.
- Also, try to display the average time taken to respond. You should use the *time* function of the *time* module to retrieve the current time.